Compiling Facts into Applications

Samuel Lippert

Abstract

A database schema, a family of state-machine workflows, and a REST API with hypermedia navigation are usually three artifacts kept in sync by hand. This paper treats them as projections of one executable object. Backus's Applicative State Transition system [1] carries a *FILE* store it never structures; Codd's relations [2] supply the structure, Object-Role Modeling [4] supplies the processes, and the representation function ρ connects facts to the functions they denote. Within a syntactically restricted fragment with stratified negation, deterministic total base operations, and acyclic value introduction, running an *application* is finite reduction over an explicit finite candidate fact space. The emitted HATEOAS surface is proved equal to the controls admitted by the operational command semantics induced by the current schema and population, so a separately authored sound and complete control map duplicates rather than adds model information. Host-registered functions form an enumerable boundary beyond which totality is not guaranteed. The object is self-similar: it extends its own schema and contains isolated copies of itself, so a tenant is a sub-store and a tenant's tenants are sub-sub-stores, each a full instance subject to the same fragment conditions. A reference implementation is available [19].

1 Foundation

Three abbreviations are overloaded terms: *AST* is Backus's Applicative State Transition [1, Sec. 14], not the compiler's syntax tree; *ORM* is Halpin's Object-Role Modeling [4], not the persistence pattern; *FP* and *FFP* are Backus's function-level systems [1, Secs. 11, 13]. The construction assumes these.

At the bottom we identify a population with its characteristic function over the declared fact domain. Under that representation, the membership question $x \in P$ and the application $P\,x$ are one act: **membership is application**. The fact base is therefore not passive data beside a program; it is the relation to which compiled objects are applied, and Backus's representation function ρ [1, Sec. 13.3], which maps an object to the function it denotes, exposes that application. A fact type $\langle CONS, s_1, \ldots, s_n \rangle$ [1, Secs. 11.2.4, 13.3.2] is a constructor of roles; populate the roles and the resulting fact is resolved by looking up its type:

$$(\rho\langle x_1, \ldots, x_n\rangle) : y = (\rho\, x_1) : \langle \langle x_1, \ldots, x_n\rangle, y\rangle. \qquad (1)$$

This is metacomposition [1, Sec. 13.3.2]. It is the only mechanism in the paper, and every later view (a constraint, a transition, a link, an API response) is an instance of it. The combining forms it composes are also Backus's [1, Sec. 11.2.4], composition $f \circ g$ and the constant $\bar{x}$, over his primitives [1, Sec. 11.2.3], the selectors and the equality test *eq*, and the empty sequence φ [1, Sec. 11.2.1].

Backus builds an AST system from three parts [1, Sec. 14.3]: an applicative subsystem (an FFP system), a state D that is its set of definitions, and transition rules carrying inputs to outputs and D to D'. The state is a sequence of cells with fetch $\uparrow n : D$ and store $\downarrow n : \langle x, D\rangle$ [1, Secs. 13.3.4, 14.3]. One cell is named *FILE* and "the system maintains" its contents, but Backus declines to structure them: "we have not said how the system's file, queries or updates are structured" [1, Sec. 14.4.4]. AREST fills exactly that hole.

Definition 1 (AREST)**.** An *Applicative REpresentational State Transfer* system is a Backus AST system [1, Sec. 14] whose *FILE* cell contains a population P (Definition 3) of a schema S (Definition 2), and whose definition cells *DEFS* hold the FFP objects [1, Sec. 13] compiled from FORML 2 readings together with those registered by the runtime. REST operations are applications of the system function (2).

A small declaration fixes the picture. In FORML 2 [18], each reading is one elementary fact:

```
Order(.OrderId) is an entity type.
Customer(.Name) is an entity type.
Order is placed by Customer.
Each Order is placed by exactly one
  Customer.
Customer ships Order.
State Machine Definition 'Order' is for
  Object Type 'Order'.
Status 'In Cart' is initial in
  State Machine Definition 'Order'.
Transition 'place' is from
  Status 'In Cart'.
Transition 'place' is to
  Status 'Placed'.
Transition 'place' is triggered by
  Fact Type 'Customer places Order'.
Transition 'ship' is from
  Status 'Placed'.
```

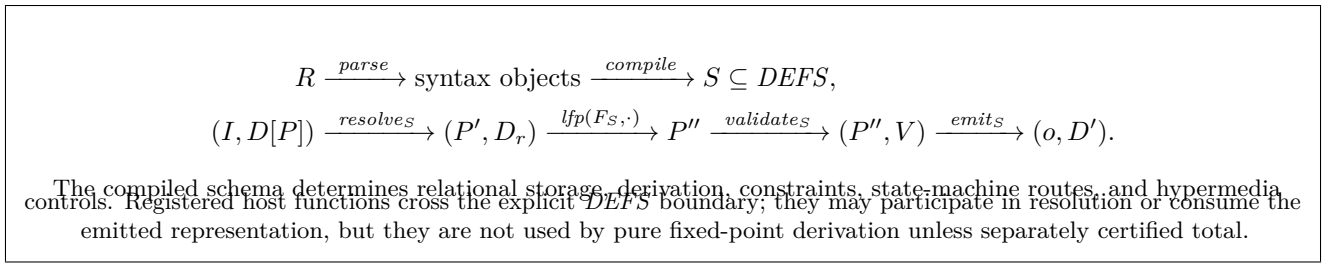


Figure 1: The AREST compilation and command pipeline. The schema, workflows, and controls remain projections of the same state object rather than independently maintained artifacts.

Command	Status after commit	Transition controls
create(o)	`In Cart`	`place`
place(c, o)	`Placed`	`ship`
ship(c, o)	`Shipped`	none

Table 1: Symbolic execution of the declaration. Navigation controls are omitted from the last column; they remain projections of the fact and uniqueness structure.

```
Transition 'ship' is to
  Status 'Shipped'.
Transition 'ship' is triggered by
  Fact Type 'Customer ships Order'.
```

Each reading is atomic, and each maps to a column: the entity references to keys, the binary fact types to foreign keys, the uniqueness reading to a restriction, and the transition facts to the machine's rows. The decomposition into atomic facts *is* the mapping to the relational schema, and together they compile to a table, a foreign key, a uniqueness restriction, and a transition that fires when its trigger fact enters P. A `POST /orders` request creates an Order and returns its status with the actions available from it; following the `place` action advances the machine to `Placed`, after which the representation offers `ship` and no longer `place`.

Table 1 is a compact witness of the synchronization claim: a committed trigger fact changes the machine population, and the next representation changes because the control query is re-evaluated over that population. No separately edited route table participates. The reference artifact [19] makes the parser, compiler, and evaluator available for inspection; the present paper makes semantic, not throughput or latency, claims.

Finite reduction. A finite population alone does not imply termination. AREST therefore admits a compiled schema only when value-introducing dependencies are acyclic, negative dependencies are stratified, and the base operations used by compiled definitions are total on their declared domains. Under those hypotheses, each system step ranges over a finite candidate fact space and derivation reaches its stratified fixed point in finitely many steps (Lemma 1). The event stream may remain unbounded over time; finiteness is required per operation over the current P. Arbitrary host functions enter only through registered definitions (Definition 10), whose termination is not guaranteed and whose extent is enumerable by Corollary 7.

2 ORM over Backus's AST

We fix the objects the rest of the paper quantifies over by reading them off ORM 2 as NORMA implements it [4, 18].

Definition 2 (Schema)**.** A *vocabulary* is a set of object types, each a *value type* (self-identifying, typed by a data type) or an *entity type* (identified by a preferred uniqueness constraint over one role, simple, or several, compound), together with a set F of fact types, each *arity*(f) roles played by object types. An entity's identifier is *auto-generated* iff its reference scheme supplies an allocator (for example, a counter, UUID, timestamp, or surrogate generator), and *supplied* otherwise. *Val* is the disjoint union of the value-type domains and the typed entity identifiers. A *schema* S comprises F, a constraint set C_S (uniqueness, mandatory, frequency, ring, value-comparison, subset, equality, exclusion, value, and cardinality constraints, each alethic or deontic [18]), a derivation-rule set R_S (Definition 7), a set of state machines, and the reference schemes. S is the compiled content of *DEFS*.

Definition 3 (Population)**.** A *population* of S is a finite consistent set

$$P \subseteq \bigcup_{f \in F} \{f\} \times Val^{arity(f)},$$

where F includes the paired explicit-negation type of each negatable fact type, and consistency excludes a ground fact and its pair from occurring together. Its finite *active domain* $adom(P)$ is the set of values occurring in P. A ground fact g is *true* in P if $g \in P$, *false* if its paired negation is in P, and *unknown* otherwise; under the closed-world assumption on an object type, unknown collapses to false. Membership is the characteristic function of P, so $g \in P$ and $P\,g$ are one act.

Entity and fact types populate *FILE*, while constraints and derivations are relational restrictions and compositions over P, and the retrieval operators are Codd's adequate θ_1: projection, natural join, tie, and restriction [2,

Sec. 2.2]. We write restriction as $\mathit{Filter}(p) : X$, the elements of a relation X satisfying a predicate p; this is Codd's θ-restriction [3, Sec. 2.3.5], with constants entering as literal relations. The equality patterns used below, compositions of *eq* with selectors and constants, reduce to the restriction of θ_1, while value ranges take the inequality θ. Indeed every constraint of Definition 2 compiles to a restriction whose predicate falls in one of two families, a cardinality count against declared bounds or a membership test against a target population (the comparison constraints taking the inequality θ), with polarity the only parameter separating symmetric from asymmetric and subset from exclusion. ORM elementary facts are in fifth normal form by construction [4], so the named relations are the cells themselves and navigation needs no separate query language. Each leg of the map is now discharged by the formalism it instantiates.

Definition 4 (Admitted fragment)**.** Let G be the keyword-delimited grammar implemented by the reference parser for the following FORML 2 sentence families: object-type and reference-scheme declarations, elementary fact-type readings, the constraints listed in Definition 2, projective derivation readings (Definition 7), and state-machine readings whose trigger is an elementary fact type. $R = L(G)$ subject to the lexical condition that a declared name contains no formal item of G as a substring. Pronoun-correlated clauses and nested objectification are outside R and are rejected. The claims below concern this admitted implementation grammar, not completeness for every FORML 2 construction.

Proposition 1 (Canonicalization of admitted readings)**.** *For every $r \in R$, $\mathit{parse}(r)$ is defined and deterministic, $\mathit{compile}(\mathit{parse}(r))$ is well defined, and $nf = \mathit{verbalize} \circ \mathit{compile} \circ \mathit{parse}$ is idempotent with $nf(r) \sim r$ for the kernel equivalence $\sim$ of $\mathit{compile} \circ \mathit{parse}$. The executable denoted by r is $\rho(\mathit{compile}(\mathit{parse}(r)))$.*

Proof. By Definition 4, R contains exactly the sentences generated by the parser's admitted families under the name condition, so parsing is defined and the keyword delimiters select one family deterministically. Compilation is functional on the resulting syntax objects. NORMA exhibits the verbalization round trip for the admitted constraint kinds [18], and *verbalize* emits the primary reading of a compiled object [5]; re-parsing therefore returns the same compiled object. Hence $nf \circ nf = nf$ and $nf(r) \sim r$. Executability is the representation function ρ [1, Sec. 13.4]. This is a canonicalization result for R, not a proof that the parser recognizes all of FORML 2. □

A state machine is itself a set of facts (a status, its transitions, and the trigger fact type of each), and advancing it is one AST step, not a second machine. Backus's transition is single: on input x the system forms $(\mathit{SYSTEM} : x)$, evaluates it under the current definitions, and obtains $\mu(\mathit{SYSTEM} : x) = \langle o, d \rangle$ with o the output and d the next state [1, Sec. 14.3.1]; the state is frozen during evaluation [1, Sec. 14.6]. Where Backus's subsystem branches on a *KEY* cell through five clauses [1, Sec. 14.4.2], AREST routes on the addressed entity,

$$\mathit{SYSTEM} : x = (\rho\,(\uparrow \mathit{entity}(x) : D)) \Uparrow \mathit{op}(x), \tag{2}$$

so new operations and new entity types extend D without disturbing one another.

Definition 5 (Command)**.** For schema S, let $\mathcal{P}_S$ and $\mathcal{D}_S$ denote its populations and states. For $I \in \mathcal{I}$ and $D \in \mathcal{D}_S$ containing $P \in \mathcal{P}_S$,

$$\begin{aligned} \mathit{resolve}_S &: \mathcal{I} \times \mathcal{D}_S \to \mathcal{P}_S \times \mathcal{D}_S, \\ \mathit{derive}_S &: \mathcal{P}_S \to \mathcal{P}_S, \\ \mathit{validate}_S &: \mathcal{P}_S \to \mathcal{P}_S \times \mathcal{P}(\mathit{Val}^*), \\ \mathit{emit}_S &: \mathcal{P}_S \times \mathcal{P}(\mathit{Val}^*) \to \mathcal{O}. \end{aligned}$$

$\mathit{resolve}_S(I, D) = \langle P', D_r \rangle$ adds the instances named by I and advances any allocator cell it uses; let $P'' = \mathit{lfp}(F_S, P')$, $\mathit{validate}_S(P'') = (P'', V)$, $\mathit{emit}_S(P'', V) = o$, and V_A be the alethic violations. Then

$$\begin{aligned} \mathit{create}_S(I, D) &= \langle o, D' \rangle, \\ D' &= \begin{cases} D_r[\mathit{FILE} \mapsto P''], & V_A = \varnothing, \\ D, & \text{otherwise.} \end{cases} \end{aligned} \tag{3}$$

Rejection rolls back both candidate facts and allocator advances.

Definition 6 (Violation)**.** For a constraint $c \in C_S$, $V_c = (\rho\, c) : P$ is the finite set of bindings that offend c, and $V = \bigcup_{c \in C_S} V_c$. An alethic c rejects the commit when $V_c \neq \varnothing$; a deontic c warns and commits. The message returned is the canonical reading of c (Proposition 1).

Definition 7 (Derivation rules)**.** A derivation rule in R_S is a role path projected onto a derived fact type: each derived role is bound to a path variable, to a calculated value (an admitted deterministic, side-effect-free total function of finitely many bound values, or an aggregate reducing a finite bag to one scalar), or to a constant [18]. An admitted calculated function returns exactly one value for each well-typed binding and neither reads nor mutates D; effects are confined to $\mathit{resolve}_S$ and post-emission registered functions. A rule is *fully derived* $(*)$ or *semi-derived* $(+$, also directly assertable), and independently *stored* (materialized) or not; completeness and storage are orthogonal, giving the four markings $*$, $**$, $+$, and $++$. Derived fact types are *projective*: no derivation rule introduces a fresh entity. The derivation dependency graph has a positive edge from a rule's derived fact type to each fact type read positively in its condition, and a negative edge to each fact type read under negation. R_S is *stratified* when no dependency cycle contains a negative edge; equivalently, each fact type can be assigned a stratum no lower than its positive dependencies and strictly higher than its negative dependencies. Negated role paths are evaluated only

against completed lower strata. Within a stratum the immediate-consequence operator is $F_S(P) = P \cup \{\text{ facts of derived fact types derivable from } P \text{ by one rule }\}$. Entity introduction is confined to $resolve_S$ and to state-transition rules, each guarded by a positive event [18].

For a finite value set A, define the typed candidate fact space

$$\mathcal{G}_S(A) = \bigcup_{f \in F} \{f\} \times A_f^{arity(f)}, \tag{4}$$

where A_f restricts each role position to the values in A of its declared object type. Because S has finitely many finite-arity fact types, $\mathcal{G}_S(A)$ is finite whenever A is finite.

Proposition 2 (Single transition)**.** *The live state-machine step is the AST transition $\mu(SYSTEM : x) = \langle o, d \rangle$. Reconstruction machine$(s_0, E)$ = foldl transition s_0 $(order_\tau E)$ is a ρ-application over the event facts, not a transition.*

Proof. A triggering event is the input x and the current status a fact in D; advancing the live machine on that event is one AST step, with D frozen throughout [1, Secs. 14.3.1, 14.6]. The function *machine* is an FFP expression over the event facts, its *foldl* derived from Backus's *while* [1, Sec. 11.2.4], evaluated within such a step, so it is a ρ-application (Proposition 3) used for migration and audit; it alone orders by each event's occurrence timestamp τ, while the live step takes events in arrival order. The two orders are Halpin's valid time and transaction time [4, Sec. 10.3]: τ is a role of the event fact itself, ordinary data, so $order_\tau$ is derivable from E, whereas arrival order is the log's and is no fact of the domain. □

Lemma 1 (Finiteness)**.** *If R_S is stratified, every base operation it uses is total on its declared domain, and no value-introducing rule lies on a cycle of its derivation dependency graph, then for every finite P the stratified fixed point $lfp(F_S, P)$ exists, equals the stratified least model of R_S above P, and is reached by iteration in finitely many steps.*

Proof. By Definition 7, projective derived fact types introduce no entity. Topologically order the acyclic subgraph of value-introducing rules. At its first node, each finite rule condition has finitely many bindings over $adom(P)$ and the schema constants; its deterministic total function returns one value per binding, and each aggregate returns one scalar per finite group. Induction along the topological order therefore adds only finitely many values. Let A be the resulting finite extension of $adom(P)$. Auto-generated identifiers remain outside F_S: $resolve_S$ or a positively guarded transition mints at most those requested by the finite current input.

Evaluate strata in increasing order. Negative premises in a stratum refer only to completed lower strata, so their anti-joins are fixed while that stratum is evaluated. The remaining immediate-consequence operator is inflationary and monotone over the finite lattice $\mathcal{P}(\mathcal{G}_S(A))$ from (4). Knaster–Tarski gives a least fixed point for each stratum [8]; Kleene iteration reaches it after at most $|\mathcal{G}_{S,k}(A)|$ strict fact additions in stratum k [9]. Induction over the finite stratum order yields the stratified least model and finite termination for the whole derivation. □

Corollary 1 (The graph hypotheses are queries)**.** *The two graph hypotheses of Lemma 1 are decidable by restrictions over D. Positive reads, negative reads, and rules' derived fact types are schema facts describing DEFS; value introduction is syntactic, so strongly connected component and reachability queries decide both conditions. Totality is an admission premise: registered definitions are enumerated by (6) and excluded from R_S unless certified total. The checks run whenever DEFS changes (Section 4) and reject a nonconforming schema like any alethic violation.*

Theorem 1 (State-transfer semantics)**.** *Let S satisfy the hypotheses of Lemma 1. For input I and state D containing P, let $\langle P', D_r \rangle = resolve_S(I, D)$, $P'' = lfp(F_S, P')$, $V = \bigcup_{c \in C_S} (\rho\, c) : P''$, and let o be the representation of $(P'', V, links)$. Then $create_S(I, D) = \langle o, D' \rangle$, where $D' = D_r[FILE \mapsto P'']$ if V has no alethic violation and $D' = D$ otherwise.*

Proof. P'' exists and is finite by Lemma 1. $validate_S$ applies each $c \in C_S$ as a restriction over P'' (Definition 6); $emit_S$ builds o from P'', V, and the links (Theorem 2). Equation (3) gives the state component. □

Definition 8 (Operationally admissible control)**.** For entity e in population P and caller c, $Adm_{S,P}(e, c)$ is the set of operation labels for which the command semantics has a well-typed route rooted at e that c may take. A label is in the set in exactly one of two ways: (i) it names a state transition whose source is $status(e)$ and whose trigger fact has a role played by e, with every other required role supplied by the request or by a uniquely determined binding; or (ii) it names a relational peer, child, or collection operation induced by a declared fact type containing e, where the target direction is determined by the applicable uniqueness constraints. In either case P must hold that c is authenticated and authorized for the label's operation on the function it addresses, and P must still hold e. Authorization is deny-by-default and granted only by facts: c is authorized for a permission on a function when P grants it to c, to an access role c has, or to an organization c belongs to (*User is authorized for Permission on Function*, asserted or derived from *Access Role is authorized for Permission on Function* and *Organization is authorized for Permission on Function*), or to any user (*Object Type has Permission*). When S declares no user, the universe of discourse has no caller to restrict and every label is authorized: an entity whose facts have been retracted affords no control. This relation is defined from $resolve_S$, the state-machine facts, the relational constraints, and the authorization facts, independently of the representation emitted by $emit_S$.

Theorem 2 (Generated-control representation)**.** *For every entity e and caller c,*

$$\begin{aligned} \mathit{links}_c(e) &= \big(\mathit{nav}(e) \cup \mathit{transitions}(\mathit{status}(e))\big) \cap \mathit{auth}_P(c) \\ &= \mathit{Adm}_{S,P}(e, c), \end{aligned} \tag{5}$$

where $\mathit{auth}_P(c)$ is the set of labels whose operation P authorizes c for, and $\mathit{links}_c(e) = \varnothing$ when P holds no fact of e. The left-hand side is a θ_1 expression over P and S. Thus the emitted controls are sound and complete for the operational command relation of Definition 8: an unauthorized control is never emitted, rather than emitted and then refused.

Proof. $\mathit{transitions}(s)$ projects exactly the transition facts restricted to source status s. Every projected label therefore has a command route of kind (i), and every route of kind (i) appears in that projection. Likewise $\mathit{nav}(e)$ examines each declared fact type in which e plays a role and projects the peer control when uniqueness spans the opposite role, the child controls when uniqueness makes the remaining roles dependent on the constrained key, and the corresponding collection controls. These are exactly the routes of kind (ii). The two cases are disjoint by their operation kind. $\mathit{auth}_P(c)$ is the union of the authorization facts whose user is c, those granted to any user, and, when S declares no user, every label; it projects their operations, so intersecting with it removes exactly the routes c may not take; and when P holds no fact of e there is no status to read and no fact type in which e plays a role, so both sides are empty. Both inclusions between $\mathit{links}_c(e)$ and $\mathit{Adm}_{S,P}(e, c)$ follow.

Each projection, and the intersection with $\mathit{auth}_P(c)$, is a restriction composing *eq* with selectors and constants followed by projection, an equality θ-restriction [3, Sec. 2.3.5] within Codd's θ_1 [2, Sec. 2.2]. No separately authored control semantics is used. □

Corollary 2 (Control-map nonredundancy)**.** *Let H be any separately authored control map over the same S and P. If, for every entity e and caller c, $H_c(e)$ contains every operationally admissible control and only operationally admissible controls, then $H_c(e) = \mathit{links}_c(e)$ for every e and c. Thus H contributes no independent model information; maintaining both H and the model duplicates the same control surface.*

Proof. By Theorem 2, $\mathit{links}_c(e)$ is exactly the set of operationally admissible controls. The two inclusions assumed for $H_c(e)$ therefore give extensional equality. □

Generating, rather than restating, this set is the novel step. Fielding [10] fixed hypermedia as a constraint but left the controls for the server author to write by hand, and prior formal treatments model the *client*'s traversal as a machine over the representations it receives [11]. Equation (5) works the other side of the wire: the generated set is current as a function of P and S, and no operationally admissible control is omitted from the generated interface. Corollary 2 states the architectural consequence: a sound and complete hand-authored control map can only repeat that projection. Adequacy of the schema to the business domain remains a modeling obligation. A client, or an agent reading the API, therefore need not invent an affordance that the declared model does not expose. A transition graph may cycle. A deontic reading may require each cycle to carry an exit transition, ensuring that no cycle is structurally closed; whether an enabled exit is eventually taken remains a fairness or domain obligation.

Corollary 3 (The view hierarchy)**.** *The uniqueness structure of Theorem 2 orders the schema: f is a* child *of g iff some fact type joins them and a uniqueness constraint spanning f's role alone makes the join functional from f to g, the mapping's foreign key. Iterated removal of parentless entity types ranks every type, types on a cycle of the child relation sharing the rank at which removal stalls. The ranking generates the* whole *view hierarchy, not an initial state: the initial representation is the collections of rank 0, a collection descends to its entities, an entity's links(e) (5) descends further, so every view is a node of the ranked hierarchy, navigation is its edge set, and both are generated as in Theorem 2, with no hand-written map. The same ranking lays out the schema itself — ranks are rows, types are cells — so the model diagram is one more view, a projection over S drawn by the machinery that draws P.*

Proposition 3 (Derivability)**.** *Every component of the representation $\mathit{repr}(e)$, namely the selectors on e's facts, the derived facts, the constraint violations, and $\mathit{links}_c(e)$ for the caller c, is $(\rho\, f) : P$ for some object f.*

Proof. Selectors, derivation rules, and constraints are ρ-applications by construction, and the links are θ_1 (Theorem 2), themselves ρ-applications. No value arises outside ρ. □

Corollary 4 (Application-policy representability)**.** *An authorization decision, validation rule, rate-limit rule, transformation, or authentication assertion is denotable as a restriction, derivation, aggregate, or fact over P whenever its inputs are represented in P and its decision function is expressible by the admitted operators and deterministic total functions of Definition 7. Policies outside that fragment remain registered definitions.*

Proof. Under the stated expressibility premise, Proposition 3 makes the result a ρ-application. Authorization may be a derivation ("a User may access a Domain iff the User belongs to an Organization that manages it"), validation is $\mathit{validate}_S$ (Theorem 1), a rate limit is a cardinality constraint over timestamped request facts, transformation is composition, and externally verified identity enters as facts through a registered boundary. The premise deliberately excludes arbitrary policies that require a partial or effectful host computation. □

This is a representability result, not an enforcement or deployment theorem. A deployment may still require proxies, queues, caches, gateways, cryptographic verification, and secret management; AREST places their application-level policy and resulting assertions in the same model rather than claiming that the supporting infrastructure disappears.

Corollary 5 (Subscription representation)**.** *A subscription can be represented as a ρ-application awaiting evaluation against a later D, and an external event as a fact entering the FILE population through $\downarrow$.*

Proof. Storing into a cell with $\downarrow$ changes D. Under a runtime binding that re-evaluates registered subscriptions on such a change, Proposition 3 makes the subscriber a ρ-application awaiting its next evaluation. A webhook, a queue message, and a sensor reading enter P by the same $\downarrow$, so externally fired and fact-fired updates are indistinguishable to the evaluator. □

This identifies the subscription semantics; delivery, buffering, retry, backpressure, and durable offsets remain responsibilities of registered platform mechanisms.

Platform binding. A runtime registers its own functions into *DEFS*, so a fact applied to one of them yields the corresponding effect:

$$(\rho\, fact) : render \to widget,$$
$$(\rho\, fact) : httpFetch \to response,$$
$$(\rho\, fact) : upsert \to D'.$$

A browser registers rendering functions, a server registers *httpFetch* and *upsert*, and one *SYSTEM* serves browser, server, and storage by varying *DEFS* rather than the logic. Binding a user interface is then registering a *render* function, so a fact renders itself. Functions that touch external state run during $resolve_S$, and those that consume the representation run after $emit_S$, which keeps $derive_S$ pure over P.

ρ and the whole-state accessor $\rho DEFS$ are Backus's [1, Secs. 13.3, 14.3.3], and the latter grants a program access to all of D "for any purpose, including the essential one of computing the successor state." Backus represents D as an object precisely because "in AST systems we shall want to transform D by applying functions to it" [1, Sec. 13.3.5]; this is the licence Section 4 uses for self-modification.

Logical deletion needs no special storage operation when lifecycle status participates in view selection: declare a terminal status deleted and compile ordinary collection and query views with a restriction excluding entities in that status. Its transition controls are then empty, while navigation may remain available in audit or administrative representations. Physical reclamation is a compaction that preserves the results of those restricted views.

Negation. ORM's negation is open-world and explicit. NORMA admits it inside derivation role paths, where a step, a root, or a branch may be negated [18], under the three-valued reading of Definition 3, verbalized "it is not true that," "it is known to be false that," and "it is not known to be false that." An epistemic falsity, verbalized "it is known to be false that," enters P as an explicit negation fact and is never inferred from absence. A negated role path is an inference from absence and evaluates as a finite anti-join against a completed lower stratum. Definition 7 excludes cycles through negative dependencies, so the anti-join is fixed while higher strata are evaluated and Lemma 1 applies. State-transition effects satisfy a separate groundedness condition: an add or delete may not occur under negation or disjunction, and each disjunctive branch must carry a positive event [18]; negation may guard a transition but never supplies its effect.

3 Cells, Tenancy, and Self-Similarity

RMAP [4] assigns each entity its own cell, the 3NF row of facts depending on its key, so D is a sequence of cells and, by [1, Sec. 14.7], "a cell in one store may contain another entire store."

Definition 9 (Cell isolation)**.** At most one step may write a given cell of D at a time, and steps writing disjoint cells may run concurrently.

The schema fixes the *shape* and maximum path depth of the dependency closure that a write may invalidate. The actual number of reachable role-player cells can grow with the current population, so this is not a constant-size recalculation claim. Re-evaluation is nevertheless confined to the written entity's cell and the cells reached through those declared dependency paths. A constraint whose scope spans cells imposes an obligation: commits touching a shared scope must serialize. The cardinality family discharges it at the constrained key itself, a single writer per scope key that generalizes Definition 9.

Consistency and availability. Modal and world-assumption declarations choose an *admissibility policy*, not a CAP classification by themselves [12]. Under a deployment that provides authoritative serialization for the relevant scope, an alethic constraint over a closed-world object type can reject a commit during a partition rather than risk the invariant; a deontic constraint over an open-world object type can accept provisional local state, warn, and reconcile when $validate_S$ (Theorem 1) runs against the authoritative P. Constraint slack can reduce the scope that requires coordination, while a bound reached at a constrained key requires serialization (Definition 9). The replication and failure protocol still determines whether the deployed service is consistent or available under partition. Likewise, the constraint set supplies an event-validity predicate, not an agreement protocol. A multi-peer implementation must separately provide ordering and agreement; hash chains can provide

tamper evidence and signatures can provide identity, but neither alone selects a canonical total order.

Tenancy. A tenant is this mechanism one level up: a cell whose contents is an entire store, so P is partitioned into per-tenant blocks.

Proposition 4 (Store-relative tenant isolation)**.** *For stores D_a and D_b in sibling tenant cells, no expression whose store access is formed from $\uparrow$ and $\downarrow$ relative to D_a denotes a cell of D_b. Isolation is preservation of store-relative addressability.*

Proof. D_b is the contents of a cell that is not a cell of D_a, and $\uparrow$ and $\downarrow$ resolve only cells of their store argument [1, Secs. 13.3.4, 14.3], so no fetch or store expression formed over D_a denotes a cell of D_b. □

Isolation is therefore a property of the shape of the state rather than a repeated predicate: within this addressing calculus, wrong-tenant access is not forbidden but unaddressable, so no additional tenant filter is required for store-relative expressions. The proposition does not constrain registered functions with ambient access, external identity, or transport; those mechanisms remain at the deployment boundary and must preserve the same capability discipline.

Sub-tenancy. The containment recurses. A tenant's store may itself hold tenant cells, each an entire store, to any finite depth, so the tenant tree is a tree of nested sub-stores. Every node is a full instance, since Corollary 6 holds per store, so a tenant carrying its own *DEFS* is an application builder who may in turn have tenants, while one inheriting its parent's is the ordinary data-isolated case. Visibility follows containment: a parent reaches into its children because their stores are cells in its own, and a child cannot address upward or sideways (Proposition 4). This is the isolation and visibility model of a platform-of-platforms; external identity, authorization, and transport controls remain deployment concerns, while tenant addressability itself follows from the nesting.

Two recursions issue from one decision. Because Backus made D a first-class object, it can be *transformed* (the system extends its own schema, Section 4) and it can *contain itself* (the nested stores above). The system evolves itself and holds isolated copies of itself, and both fall out of the same move.

4 Closure and the Enumerable Boundary

Schema extension is an ordinary step: the addressed entity is *DEFS* and the operation is $\mathit{compile} \circ \mathit{parse}$, with admitted new objects entering through $\downarrow \mathit{DEFS}$.

Corollary 6 (Closure under admitted extension)**.** *A schema-extension step changes only DEFS and the schema facts describing it; the evaluator, parser, compiler, verbalizer, and registered functions are unchanged. After a step whose operation is $\mathit{compile} \circ \mathit{parse}$ on readings $R' \subseteq R$, let S' be the resulting schema. Provided its definitions pass the admission checks of Corollary 1 and $\mathit{validate}_{S'}$ reports no alethic violation in the current population, Definitions 2–7 and the results through Theorem 2 hold for every later step over D'. This corollary covers extensions that preserve the interpretation of existing declarations; removal or reinterpretation requires an explicit migration and is not claimed here.*

Proof. No statement's argument depends on D being fixed: Proposition 1 rests on the grammar and the kernel quotient, while the others range over P and S, which now include the extension. Corollary 1 re-establishes the graph hypotheses, while admission re-establishes totality, for the resulting *DEFS*. Ingestion is itself a *create* step, so it is subject to $\mathit{validate}_{S'}$ (Theorem 1): an extension whose alethic constraints the current P violates is rejected, so a migration stages as derivations or deontic rules until P complies. □

This is what makes a tenant's own admitted schema extension (Section 3) safe, and it is the first of the two recursions. The second is the boundary at which the guarantees stop.

Definition 10 (Registered function)**.** A definition in *DEFS* is a tuple $\langle \mathit{name}, \mathit{dom}, \mathit{cod}, \mathit{origin}, \mathit{impl} \rangle$ with $\mathit{origin} \in \{\mathit{compiled}, \mathit{registered}\}$. A *compiled* definition has $\mathit{impl} = \rho(o)$ for an object o admitted by Definition 4: it depends only on admitted total base operations and compiled definitions, its negative dependencies are stratified, and value-introducing recursion is excluded by Lemma 1. Its evaluation is therefore total for finite P. A *registered* definition has *impl* supplied by the host runtime and is in general partial.

Corollary 7 (Enumerable host boundary)**.** *The host-supplied semantic surface of the system is the restriction*

$$\mathit{Filter}(\mathit{eq} \circ [s_{\mathit{origin}}, \overline{\mathit{registered}}]) : \mathit{DEFS}, \tag{6}$$

a ρ-application (Proposition 3), and it marks the boundary at which unverified partial computation may enter.

Proof. By Definition 10, an admitted compiled definition is total for finite P, whereas a registered definition is arbitrary host code whose termination is not guaranteed. The restriction (6) selects exactly those host-supplied definitions, the point at which unchecked general computation may re-enter the semantics. □

An object type backed by an external system sits on this line: its population is fetched by a registered function, and its facts enter under the open-world assumption [4]. A deontic constraint over such facts is sound but not complete: a reported violation is genuine, while under the open world the absence of one guarantees nothing. Host extension is not removed but localized to a queryable

fact set. The parser, compiler, evaluator, and verbalizer remain a fixed trusted kernel; Equation (6) enumerates the variable host extensions rather than the kernel itself.

A registered definition that shadows a compiled definition of the same name contributes no additional semantic boundary only when its equivalence to the compiled object is established by a trusted rewrite or certificate. In that case the shadow may be dropped without changing any value; without such evidence it remains selected by Equation (6). Backus's algebra of programs [1, Sec. 12.2] licenses the certified case: its laws are equations between the forms represented by compiled objects, so an object may be replaced, ahead of time or at registration, by an equivalently certified form.

5 Related Work

Deductive databases and Datalog give recursive rules over relations a least-fixed-point semantics [13]. AREST uses the same finite-lattice shape for within-stratum derivation, but places the population inside Backus's state object and restricts calculated values, negation, and entity introduction so that the candidate fact space of each command is explicit. Active database systems connect database events to rules and actions [14]; AREST instead represents event and machine declarations as facts, keeps $derive_S$ pure, and confines effects to resolution or registered platform mechanisms.

Model-driven engineering generates software artifacts from higher-level models [15], including formal transformations from object models to data and behavioral components [16]. AREST makes a narrower claim: the relational layout, transition routes, and hypermedia controls remain projections of one runtime object, and Theorem 2 identifies the generated controls with an independently stated command relation. It does not claim that every deployment concern is generated or decidable.

The store-relative isolation result is related to protection by structured naming and addressability in capability-oriented systems [17]. Proposition 4 specializes that idea to nested AST stores: sibling stores are absent from one another's address space. The result does not cover registered functions with ambient authority, identity protocols, or transport enforcement.

6 Conclusion

The gap between domain knowledge and running software, where requirements drift and production breaks, is usually treated as inevitable. AREST narrows that gap by compiling the schema, executable readings, state transitions, and generated hypermedia controls from one fact model. Within the admitted fragment, each representation value is a ρ-application over P (Proposition 3), derivation terminates under the explicit conditions of Lemma 1, and every operationally admissible control is generated from the current schema and population (Theorem 2), making a separately maintained sound and complete control map redundant (Corollary 2). This establishes the hypermedia side of REST; stateless request semantics, cache directives, layered deployment, and client-executed code remain separate protocol and deployment obligations [10]. The operational surface still maps naturally into the object: versioning is represented by the event stream, migration is ingestion staged by Corollary 6, and auditability is the derivation chain. The fixed trusted kernel remains an implementation assumption, while variable host computation is enumerable (Corollary 7). The result is not that all software becomes decidable, but that the decidable compiled fragment and the boundary beyond it are stated explicitly and can be inspected by the system itself.

Acknowledgments

LLMs assisted with implementation and manuscript preparation. The architectural decisions and the composition of FFP/AST with REST are the author's.

References

[1] J. Backus, "Can Programming Be Liberated from the von Neumann Style? A Functional Style and Its Algebra of Programs," *Commun. ACM*, 21(8):613–641, 1978.

[2] E.F. Codd, "A Relational Model of Data for Large Shared Data Banks," *Commun. ACM*, 13(6):377–387, 1970.

[3] E.F. Codd, "Relational Completeness of Data Base Sublanguages," in *Data Base Systems*, Courant Computer Science Symposia 6, Prentice-Hall, 1972, pp. 65–98.

[4] T. Halpin and T. Morgan, *Information Modeling and Relational Databases*, 2nd ed., Morgan Kaufmann, 2008.

[5] T. Halpin and M. Curland, "Automated Verbalization for ORM 2," in *OTM Workshops*, pp. 1181–1190, 2006.

[6] M.H. van Emden and R.A. Kowalski, "The Semantics of Predicate Logic as a Programming Language," *J. ACM*, 23(4):733–742, 1976.

[7] D.S. Scott, "Data Types as Lattices," *SIAM J. Comput.*, 5(3):522–587, 1976.

[8] A. Tarski, "A lattice-theoretical fixpoint theorem and its applications," *Pacific J. Math.*, 5(2):285–309, 1955.

[9] S.C. Kleene, *Introduction to Metamathematics*, North-Holland, 1952.

[10] R.T. Fielding, "Architectural Styles and the Design of Network-based Software Architectures," Ph.D. dissertation, Univ. of California, Irvine, 2000.

[11] I. Zužak, I. Budiselić, and G. Delač, "Formal Modeling of RESTful Systems Using Finite-State Machines," in *Proc. ICWE*, LNCS 6757, pp. 346–360, 2011.

[12] S. Gilbert and N. Lynch, "Brewer's Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services," *ACM SIGACT News*, 33(2):51–59, 2002.

[13] S. Ceri, G. Gottlob, and L. Tanca, "What You Always Wanted to Know About Datalog (And Never Dared to Ask)," *IEEE Trans. Knowl. Data Eng.*, 1(1):146–166, 1989.

[14] J. Widom and S. Ceri, eds., *Active Database Systems: Triggers and Rules for Advanced Database Processing*. Morgan Kaufmann, 1996.

[15] D.C. Schmidt, "Guest Editor's Introduction: Model-Driven Engineering," *Computer*, 39(2):25–31, 2006.

[16] C.-W. Wang and J. Davies, "Formal Model-Driven Engineering: Generating Data and Behavioural Components," *Electron. Proc. Theor. Comput. Sci.*, 105:100–117, 2012.

[17] J.B. Dennis and E.C. Van Horn, "Programming Semantics for Multiprogrammed Computations," *Commun. ACM*, 9(3):143–155, 1966.

[18] T. Halpin and M. Curland, NORMA (Natural Object-Role Modeling Architect), open-source reference implementation, `https://github.com/ormsolutions/NORMA`.

[19] S. Lippert, AREST, open-source reference implementation, `https://github.com/sam-lippert/arest`.